



React JS

Complete Handwritten-Style Notes

From Basics to Advanced • Hooks • Router • Redux Toolkit • Full-Stack with Spring Boot • Interview Prep

★ **Remember:** A complete, exam & interview ready notebook covering JSX, Components, Hooks, Routing, Redux Toolkit, API integration, Auth, Testing, Deployment, 200+ curated interview Q&A, coding exercises and 5 mini projects.

■ Table of Contents

1. Introduction to React
2. Environment Setup
3. JSX (JavaScript XML)
4. Components
5. Props
6. State
7. Event Handling
8. Conditional Rendering
9. Lists & Keys
10. Forms
11. Hooks - Complete Guide
12. Routing (React Router)
13. API Integration
14. Context API
15. Redux Toolkit (RTK)
16. Styling in React
17. Performance Optimization
18. Authentication
19. React with Spring Boot (Full-Stack)
20. Testing
21. Deployment
22. React Interview Questions (curated)
23. Coding Practice Questions
24. Mini Projects
25. Cheat Sheet

CH 1

Introduction to React

◆ What is React?

React is an open-source JavaScript library (not a full framework) created by Facebook (Meta) for building fast, interactive user interfaces, mainly for single-page applications. It lets you build encapsulated 'components' that manage their own state, then compose them into complex UIs.

- Created by Jordan Walke, released by Facebook in 2013.
- Declarative: you describe WHAT the UI should look like, React figures out HOW to update the DOM.
- Component-based: UI is broken into small, reusable, independent pieces.
- Learn once, write anywhere: React Native uses the same concepts for mobile apps.

◆ Key Features

- JSX - HTML-like syntax inside JavaScript.
- Virtual DOM - fast, efficient re-rendering.
- One-way (unidirectional) data binding - data flows from parent to child via props.
- Component-based architecture - functional & class components.
- Hooks - use state and lifecycle features in function components.
- Rich ecosystem - React Router, Redux, React Query, Next.js, etc.

◆ Advantages

- Reusable components speed up development.
- Virtual DOM makes updates fast.
- Huge community & ecosystem, backed by Meta.
- SEO friendly when combined with SSR (Next.js).
- Easy to learn if you know JavaScript.

◆ Disadvantages

- Only a UI library - routing, state management, HTTP need extra libraries.
- JSX has a learning curve for beginners.
- Fast pace of ecosystem changes.
- SEO for pure client-side rendered apps needs extra setup.

◆ Virtual DOM

The Virtual DOM (VDOM) is a lightweight, in-memory JavaScript representation of the real DOM. When state changes, React creates a new virtual DOM tree, compares (diffs) it with the previous one using the 'reconciliation' algorithm, and then updates only the changed parts of the real DOM

(batched, efficient updates).

1. State changes -> new Virtual DOM tree is created.
2. React diffs new tree with previous tree (diffing algorithm).
3. React calculates the minimal set of changes needed.
4. React updates only those nodes in the real DOM (reconciliation).

◆ ***Real DOM vs Virtual DOM***

Real DOM	Virtual DOM
Updates are slow, re-renders full tree	Updates are fast, in-memory diffing
Direct manipulation is expensive	Manipulation is cheap (JS object)
Creates a new DOM if element updates	Updates only changed elements
Can cause memory wastage	Efficient memory usage

◆ ***SPA vs MPA***

Single Page Application (SPA)	Multi Page Application (MPA)
One HTML page, content updates dynamically	Every action loads a new HTML page
Faster after initial load	Slower - full reload each time
React, Angular, Vue based	Traditional PHP/JSP/server rendered sites
Harder default SEO	Better default SEO

◆ ***React Architecture (High Level)***

- Components - building blocks of UI (functional/class).
- JSX - syntax to describe UI.
- Virtual DOM - reconciliation engine.
- React Fiber - the reconciliation algorithm/engine since React 16, enables incremental rendering.
- Renderer (ReactDOM) - renders components into the actual DOM (or React Native renders to native views).

CH 2

Environment Setup

◆ **Node.js & npm**

Node.js is a JavaScript runtime needed to run build tools. npm (Node Package Manager) installs and manages packages/libraries. npx runs a package binary without installing it globally - used to scaffold projects.

```
code
node -v
npm -v
npx -v
```

◆ **Creating a Project - Vite (Recommended, fast)**

```
code
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

◆ **Creating a Project - Create React App (legacy)**

```
code
npx create-react-app my-app
cd my-app
npm start
```

◆ **Typical Project Structure**

```
code
my-app/
  node_modules/
  public/
    index.html
  src/
    assets/
    components/
    pages/
    hooks/
    context/
    App.jsx
    main.jsx
  package.json
  vite.config.js
```

★ **Remember:** *package.json* tracks dependencies + scripts. Never edit *node_modules* directly. *.gitignore* should exclude *node_modules* and *.env* files.

◆ Common npm scripts

Command	Purpose
npm run dev	Start dev server (Vite)
npm start	Start dev server (CRA)
npm run build	Production build
npm run preview	Preview production build
npm install <pkg>	Add a dependency
npm install -D <pkg>	Add a dev dependency

CH 3

JSX (JavaScript XML)

◆ **What is JSX?**

JSX lets you write HTML-like markup directly inside JavaScript. It is syntactic sugar for `React.createElement()` calls, compiled by Babel.

```
code
const element = <h1>Hello, World!</h1>;

// Compiles roughly to:
const element = React.createElement('h1', null, 'Hello, World!');
```

◆ **Embedding Expressions**

Use curly braces `{}` to embed any valid JS expression - variables, function calls, ternary operators, arithmetic.

```
code
const name = 'Aditi';
const el = <h1>Hello, {name}! You are {2024 - 2000} years old.</h1>;
```

◆ **JSX Rules**

- Must return a SINGLE root element (or use a Fragment `<>...</>`).
- Use `className` instead of `class` (`class` is a reserved JS keyword).
- Use `htmlFor` instead of `for` (in `<label>`).
- All tags must be closed - `<img />`, `<br />`.
- camelCase for attributes: `onClick`, `tabIndex`, `strokeWidth`.
- JSX expressions must evaluate to a value - `if/else` statements are NOT allowed directly, use ternary or `&&`.

◆ **Fragments**

Fragments group a list of children without adding an extra node to the DOM.

```
code
function List() {
  return (
    <>
      <li>Item 1</li>
      <li>Item 2</li>
    </>
  );
}
```

◆ **Inline Styling in JSX**

code

```
const style = { color: 'blue', fontSize: '20px' };  
const el = <p style={style}>Styled text</p>;  
// or inline  
<p style={{ color: 'red' }}>Red text</p>
```

CH 4

Components

◆ **Functional Components (modern, preferred)**

A functional component is simply a JavaScript function that returns JSX. Since Hooks (React 16.8), functional components can hold state and use lifecycle-like behaviour.

```
code
function Welcome({ name }) {
  return <h1>Hello, {name}</h1>;
}

export default Welcome;
```

◆ **Class Components (legacy)**

```
code
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

◆ **Component Lifecycle (Class Components)**

Phase	Methods
Mounting	constructor -> render -> componentDidMount
Updating	shouldComponentUpdate -> render -> componentDidUpdate
Unmounting	componentWillUnmount
Error handling	componentDidCatch, getDerivedStateFromError

★ **Remember:** In function components the same lifecycle behaviour is achieved with `useEffect`: mount -> effect runs once (`[]`); update -> effect runs on dependency change; unmount -> the cleanup function returned from `useEffect`.

◆ **Functional vs Class Components**

Functional Component	Class Component
Plain JS function	ES6 class extending <code>React.Component</code>
State via <code>useState/useReducer</code> hooks	State via <code>this.state</code> / <code>this.setState</code>
Lifecycle via <code>useEffect</code>	Lifecycle methods (<code>componentDidMount</code> etc.)

Functional Component	Class Component
Less boilerplate, easier to test	More boilerplate, 'this' binding needed
Recommended in modern React	Still supported, mostly legacy code

◆ ***Component Best Practices***

- One component = one responsibility (Single Responsibility Principle).
- Keep components small and composable.
- Name components in PascalCase; files match component names.
- Lift state up only as far as necessary.
- Destructure props for readability.
- Prefer functional components + hooks for new code.

CH 5

Props

◆ **What are Props?**

Props (properties) are read-only inputs passed from a parent component to a child component. They make components reusable and configurable, and enforce a one-way (top-down) data flow.

```
code
function Greeting(props) {
  return <h2>Hello, {props.name}!</h2>;
}

function App() {
  return <Greeting name="Rahul" />;
}
```

◆ **Destructuring Props**

```
code
function Greeting({ name, age }) {
  return <h2>{name} is {age} years old</h2>;
}
```

◆ **children Prop**

props.children lets a component render whatever is nested between its opening and closing tags - useful for wrapper/layout components.

```
code
function Card({ children }) {
  return <div className="card">{children}</div>;
}

<Card>
  <h3>Title</h3>
  <p>Body text inside the card</p>
</Card>
```

◆ **Default Props**

```
code
function Button({ label = 'Click Me' }) {
  return <button>{label}</button>;
}
// or (older syntax)
Button.defaultProps = { label: 'Click Me' };
```

◆ **PropTypes (runtime type checking)**

```
code
import PropTypes from 'prop-types';

function User({ name, age }) {
  return <p>{name} - {age}</p>;
}

User.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
};
```

★ **Remember:** Props are IMMUTABLE - a child must never modify props directly. If a child needs to change data, the parent should pass down a callback function as a prop.

CH 6

State

◆ **What is State?**

State is data that is local to a component and can change over time (e.g. via user interaction). Unlike props, state is managed within the component and updating it triggers a re-render.

◆ **useState Hook**

```
code
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

◆ **Functional Updates**

When new state depends on the previous state, pass a function to the setter to avoid stale-state bugs, especially in loops, timers, or rapid clicks.

```
code
setCount(prevCount => prevCount + 1);
```

◆ **State with Objects/Arrays - Immutability**

Never mutate state directly. Always create a new object/array so React detects the change and re-renders.

```
code
const [user, setUser] = useState({ name: 'A', age: 20 });

// WRONG - mutates directly
// user.age = 21;

// CORRECT - spread into a new object
setUser(prev => ({ ...prev, age: 21 }));

const [items, setItems] = useState([1, 2, 3]);
setItems(prev => [...prev, 4]); // add
setItems(prev => prev.filter(i => i !== 2)); // remove
```

◆ **Props vs State**

Props	State
Passed from parent (external)	Managed within component (internal)
Read-only / immutable	Mutable via setState/useState setter
Used to configure a component	Used to track dynamic data
Changes trigger re-render of child	Changes trigger re-render of owner

CH 7

Event Handling

◆ **Handling Events in React**

React wraps native DOM events into SyntheticEvent - a cross-browser wrapper with the same interface as native events. Event handler names use camelCase and are passed as functions (not strings).

```
code
function Button() {
  function handleClick(e) {
    console.log('Button clicked', e);
  }
  return <button onClick={handleClick}>Click Me</button>;
}
```

◆ **Passing Arguments to Handlers**

```
code
function Item({ id, onDelete }) {
  return <button onClick={() => onDelete(id)}>Delete</button>;
}
```

◆ **Common Events**

Event	Use case
onClick	Button/element clicks
onChange	Input/select value changes
onSubmit	Form submission
onMouseEnter/Leave	Hover interactions
onKeyDown/onKeyUp	Keyboard interactions
onFocus/onBlur	Input focus tracking

◆ **preventDefault & stopPropagation**

```
code
function Form() {
  function handleSubmit(e) {
    e.preventDefault(); // stop page reload
    console.log('submitted');
  }
  return <form onSubmit={handleSubmit}>...</form>;
}
```

★ **Remember:** `e.stopPropagation()` stops an event from bubbling up to parent elements' handlers.

CH 8

Conditional Rendering

◆ **Techniques**

Since JSX is just JavaScript, you use normal JS expressions (not if/else statements directly inside JSX) to decide what to render.

◆ **1. if / else (outside JSX, before return)**

```
code
function Status({ loggedIn }) {
  if (loggedIn) {
    return <p>Welcome back!</p>;
  }
  return <p>Please log in.</p>;
}
```

◆ **2. Ternary Operator (inline, for two outcomes)**

```
code
<p>{loggedIn ? 'Welcome back!' : 'Please log in.'}</p>
```

◆ **3. Logical && (render or render nothing)**

```
code
{unreadCount > 0 && <span className="badge">{unreadCount}</span>}
```

★ **Remember:** Careful: `{count && <div/>}` renders '0' if count is 0, since 0 is falsy but still gets printed. Use `{count > 0 && ...}` instead.

◆ **4. Switch / Object Map (multiple outcomes)**

```
code
function StatusBadge({ status }) {
  const map = {
    success: <span className="green">Success</span>,
    error: <span className="red">Error</span>,
    pending: <span className="grey">Pending</span>,
  };
  return map[status] || null;
}
```

CH 9

Lists & Keys

◆ **Rendering Lists**

Use `Array.prototype.map()` to transform an array of data into an array of JSX elements.

```
code
const fruits = ['Apple', 'Banana', 'Mango'];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

◆ **Why Keys Matter**

Keys give each list item a stable identity so React's diffing algorithm can correctly track which items were added, removed, or reordered - instead of re-rendering the whole list.

- Keys must be unique among siblings (not globally unique).
- Prefer a stable, unique ID from your data (e.g. `item.id`) over array index.
- Using the index as key can cause bugs when the list is reordered, filtered, or items are inserted/removed (state can get mixed up between items).
- Keys are not passed down as props to the child component.

```
code
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}
```

CH 10

Forms

◆ **Controlled Components (recommended)**

In a controlled component, form data is handled by React state - the input's value is always driven by state, and onChange updates that state. React is the 'single source of truth'.

code

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    console.log(email, password);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

◆ **Uncontrolled Components**

The DOM itself keeps track of the form data. You read the value only when needed, using a ref, instead of on every keystroke.

```
code
function UncontrolledForm() {
  const inputRef = useRef();

  function handleSubmit(e) {
    e.preventDefault();
    console.log(inputRef.current.value);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} defaultValue="" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

◆ ***Controlled vs Uncontrolled***

Controlled	Uncontrolled
Value driven by React state	Value stored in the DOM
Uses value + onChange	Uses ref + defaultValue
Easier validation, instant feedback	Less code, closer to plain HTML
More re-renders	Fewer re-renders

◆ ***Popular Form Libraries***

- React Hook Form - minimal re-renders, easy validation.
- Formik - widely used, integrates with Yup for schema validation.

CH 11

Hooks - Complete Guide

◆ **Rules of Hooks**

- Only call hooks at the TOP LEVEL of a function component (not inside loops, conditions, or nested functions).
- Only call hooks from React function components or custom hooks.
- Hook call order must stay consistent across renders.

◆ **useState**

Adds local state to a function component.

```
code
const [count, setCount] = useState(0);
```

◆ **useEffect - side effects**

Runs side effects (data fetching, subscriptions, timers, manual DOM changes) after render. The dependency array controls when it re-runs.

```
code
useEffect(() => {
  console.log('runs after every render');
});

useEffect(() => {
  console.log('runs only once, on mount');
}, []);

useEffect(() => {
  console.log('runs when count changes');
}, [count]);

useEffect(() => {
  const timer = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(timer); // cleanup on unmount
}, []);
```

◆ **useContext - avoid prop drilling**

```
code
const ThemeContext = createContext('light');

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>Toolbar</div>;
}
```

◆ *useReducer - complex state logic*

An alternative to useState for state that involves multiple sub-values or complex transitions - similar to Redux's reducer pattern.

```
code
function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <>
    <p>{state.count}</p>
    <button onClick={() => dispatch({ type: 'increment' })}>+</button>
    <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </>
  );
}
```

◆ *useMemo - memoize expensive calculations*

```
code
const total = useMemo(() => {
  return items.reduce((sum, i) => sum + i.price, 0);
}, [items]);
```

◆ *useCallback - memoize functions*

```
code
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

★ **Remember:** *useMemo* memoizes a **VALUE**. *useCallback* memoizes a **FUNCTION**. Both prevent unnecessary recalculation/re-creation, and help avoid needless re-renders of memoized child components (React.memo).

◆ *useRef - persist values without re-render*

```
code
function TextInput() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);
  return <input ref={inputRef} />;
}
```

- Access/mutate DOM nodes directly (inputRef.current).

- Store a mutable value that persists across renders WITHOUT causing a re-render (e.g. timer IDs, previous values).

◆ Custom Hooks

A custom hook is just a JS function whose name starts with 'use' and that calls other hooks - used to extract and reuse stateful logic across components.

```
code
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width;
}

// usage
function App() {
  const width = useWindowWidth();
  return <p>Window width: {width}</p>;
}
```

◆ Hooks Quick Reference

Hook	Purpose
useState	Local component state
useEffect	Side effects / lifecycle
useContext	Consume context, avoid prop drilling
useReducer	Complex state transitions
useMemo	Memoize computed values
useCallback	Memoize function references
useRef	Mutable ref / direct DOM access
useLayoutEffect	Sync effect before browser paints
useId	Generate unique, stable IDs (SSR safe)

CH 12

Routing (React Router)

◆ Installation & Setup

```
code
npm install react-router-dom
```

```
code
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

◆ Navigation

```
code
import { Link, NavLink, useNavigate } from 'react-router-dom';

<Link to="/about">About</Link>
<NavLink to="/" className={({isActive}) => isActive ? 'active' : ''}>Home</NavLink>

function Component() {
  const navigate = useNavigate();
  return <button onClick={() => navigate('/login')}>Go to Login</button>;
}
```

◆ Dynamic Routing / URL Params

```
code
import { useParams } from 'react-router-dom';

function UserDetail() {
  const { id } = useParams();
  return <p>User ID: {id}</p>;
}
```

◆ Query Params

code

```
import { useSearchParams } from 'react-router-dom';

function Search() {
  const [params, setParams] = useSearchParams();
  const query = params.get('q');
  return <p>Searching for: {query}</p>;
}
```

◆ Nested Routes

code

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>

// Inside Dashboard component:
import { Outlet } from 'react-router-dom';
function Dashboard() {
  return (
    <div>
      <Sidebar />
      <Outlet /> {/* renders matched child route */}
    </div>
  );
}
```

◆ Protected Routes

code

```
function ProtectedRoute({ children }) {
  const isAuthenticated = Boolean(localStorage.getItem('token'));
  return isAuthenticated ? children : <Navigate to="/login" replace />;
}

<Route path="/dashboard" element={
  <ProtectedRoute><Dashboard /></ProtectedRoute>
} />
```

CH 13

API Integration

◆ Fetch API

code

```
useEffect(() => {
  async function loadData() {
    try {
      setLoading(true);
      const res = await fetch('https://api.example.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      const data = await res.json();
      setUsers(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
  loadData();
}, []);
```

◆ Axios

code

```
npm install axios
```

code

```
import axios from 'axios';

const api = axios.create({ baseURL: 'https://api.example.com' });

// GET
const res = await api.get('/users');

// POST
await api.post('/users', { name: 'Sam' });

// with interceptor for auth token
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

◆ Loading / Error / Empty State Pattern

code

```
function UserList() {  
  const [users, setUsers] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  // ...fetch logic in useEffect  
  
  if (loading) return <p>Loading...</p>;  
  if (error) return <p>Error: {error}</p>;  
  if (users.length === 0) return <p>No users found</p>;  
  
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;  
}
```

◆ Fetch vs Axios

fetch	axios
Built into browsers, no install	External library
Needs manual res.json() and error check	Auto JSON parsing, throws on HTTP errors
No built-in interceptors	Interceptors for request/response
No automatic request cancellation (old)	Built-in cancel tokens / AbortController support

CH 14

Context API

◆ Why Context?

Context lets you share data (theme, auth user, language, etc.) across the component tree WITHOUT manually passing props down through every level (avoids 'prop drilling').

◆ Creating & Using Context

code

```
// AuthContext.js
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => setUser(userData);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

code

```
// main.jsx
<AuthProvider>
  <App />
</AuthProvider>

// any nested component
function Navbar() {
  const { user, logout } = useAuth();
  return user ? <button onClick={logout}>Logout {user.name}</button> : null;
}
```

★ **Remember:** Context is great for low-frequency-updating global data (theme, auth, locale). For frequently-changing, complex state, a dedicated state manager (Redux, Zustand) usually performs better.

CH 15

Redux Toolkit (RTK)

◆ **Why Redux Toolkit?**

Redux Toolkit is the official, recommended way to write Redux logic. It reduces boilerplate, includes Immer (write 'mutating' code that produces immutable updates), and bundles good defaults (devtools, thunk middleware).

```
code
npm install @reduxjs/toolkit react-redux
```

◆ **Store**

```
code
// store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

export const store = configureStore({
  reducer: { counter: counterReducer },
});
```

◆ **Slice (state + reducers + actions in one place)**

```
code
// counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; }, // Immer handles immutability
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

◆ **Connecting to React**

code

```
// main.jsx
import { Provider } from 'react-redux';
import { store } from './store';

<Provider store={store}>
  <App />
</Provider>
```

code

```
// Counter.jsx
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from './counterSlice';

function Counter() {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();
  return (
    <>
      <p>{count}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
    </>
  );
}
```

◆ Async Logic - createAsyncThunk

code

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
import axios from 'axios';

export const fetchUsers = createAsyncThunk('users/fetch', async () => {
  const res = await axios.get('/api/users');
  return res.data;
});

const userSlice = createSlice({
  name: 'users',
  initialState: { list: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      });
  },
});
```

CH 16

Styling in React

◆ Plain CSS

```
code
import './App.css';
<div className="container">Hello</div>
```

◆ CSS Modules (scoped styles)

```
code
/* Button.module.css */
.btn { background: blue; color: white; }

// Button.jsx
import styles from './Button.module.css';
<button className={styles.btn}>Click</button>
```

◆ Sass / SCSS

```
code
npm install -D sass
```

```
code
// styles.scss
$primary: #4f46e5;
.card {
  padding: 1rem;
  &:hover { border-color: $primary; }
}
```

◆ Tailwind CSS (utility-first)

```
code
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

```
code
<button className="px-4 py-2 bg-indigo-600 text-white rounded-lg hover:bg-indigo-700">
  Click Me
</button>
```

◆ Styled Components (CSS-in-JS)

```
code
npm install styled-components
```

```
code
import styled from 'styled-components';

const Button = styled.button`
  background: #4f46e5;
  color: white;
  padding: 10px 16px;
  border-radius: 8px;
  &:hover { background: #4338ca; }
`;

<Button>Click Me</Button>
```

◆ Choosing an Approach

Approach	Best for
Plain CSS	Small apps, simplicity
CSS Modules	Component-scoped styles, no library needed
Tailwind CSS	Rapid UI building, consistent design system
Styled Components	Dynamic, prop-driven styling, theming

CH 17

Performance Optimization

◆ **React.memo - skip re-render if props unchanged**

```
code
const UserCard = React.memo(function UserCard({ user }) {
  console.log('rendered', user.name);
  return <div>{user.name}</div>;
});
```

◆ **useMemo & useCallback**

Covered in Chapter 11 - memoize expensive computations and stable function references so children wrapped in React.memo don't re-render unnecessarily.

◆ **Lazy Loading Components**

```
code
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <Dashboard />
    </Suspense>
  );
}
```

◆ **Code Splitting**

Bundlers (Vite/Webpack) automatically split code into separate chunks at each dynamic import(), so users only download the JS needed for the current page/route - route-based code splitting is the most common pattern.

◆ **Other Optimization Techniques**

- Virtualize long lists (react-window / react-virtualized) - render only visible rows.
- Debounce/throttle expensive handlers (search input, resize, scroll).
- Avoid inline object/array/function literals passed as props when a child is memoized (they create a new reference every render).
- Use production build (npm run build) - dev build includes extra warnings/checks and is much slower.
- Use key correctly in lists to avoid unnecessary unmount/remount.
- Analyze bundle with tools like source-map-explorer / Vite's rollup-plugin-visualizer.

CH 18

Authentication

◆ **JWT (JSON Web Token) Basics**

A JWT is a compact, signed token (header.payload.signature) issued by the server after successful login. The client stores it and sends it on every subsequent request to prove identity, avoiding the need to re-send credentials.

◆ **Typical Login Flow**

1. User submits email/password via a controlled login form.
2. Frontend POSTs credentials to the backend /login endpoint.
3. Backend verifies credentials, generates a JWT (access token, and often a refresh token).
4. Frontend stores the token (memory / httpOnly cookie is safest; localStorage is common but vulnerable to XSS).
5. Frontend attaches the token as an Authorization: Bearer <token> header on future API calls.
6. Backend middleware verifies the token on protected routes and identifies the user.

```
code
// login
const res = await axios.post('/api/login', { email, password });
localStorage.setItem('token', res.data.token);

// attaching token to requests
axios.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

◆ **Protected Routes (recap)**

```
code
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" />;
}
```

★ **Remember:** Never store sensitive data in plain localStorage if you can avoid it; prefer httpOnly, secure cookies set by the server to reduce XSS risk. Always validate/verify tokens server-side - never trust the client.

CH 19

React with Spring Boot (Full-Stack)

◆ **Typical Architecture**

React (frontend, port 5173/3000) communicates with a Spring Boot REST API (backend, port 8080) over HTTP/JSON. In development, a proxy or CORS configuration bridges the two different origins.

◆ **Enabling CORS on Spring Boot**

```
code
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5173")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

◆ **CRUD from React**

```
code
const API = 'http://localhost:8080/api/employees';

// CREATE
await axios.post(API, { name, department });

// READ
const { data } = await axios.get(API);

// UPDATE
await axios.put(`${API}/${id}`, { name, department });

// DELETE
await axios.delete(`${API}/${id}`);
```

◆ **JWT Auth with Spring Security**

- Spring Boot generates a JWT on /auth/login using Spring Security + jjwt library.
- React stores the token and sends it as Authorization: Bearer <token>.
- A JwtAuthenticationFilter on the backend validates the token on each request and sets the SecurityContext.
- Backend endpoints are secured with @PreAuthorize or SecurityFilterChain rules.

◆ **Deployment Note**

In production, either serve the React build as static files from Spring Boot's /static folder (single deployable jar), or deploy the React app separately (Vercel/Netlify) and point it at the Spring Boot API's public URL.

CH 20

Testing

◆ Jest

Jest is a JavaScript testing framework - test runner, assertion library, and mocking utilities all in one. Comes pre-configured with Create React App.

```
code
npm install -D jest @testing-library/react @testing-library/jest-dom
```

```
code
// sum.js
export function sum(a, b) { return a + b; }

// sum.test.js
import { sum } from './sum';

test('adds 2 + 3 to equal 5', () => {
  expect(sum(2, 3)).toBe(5);
});
```

◆ React Testing Library (RTL)

RTL encourages testing components the way a user interacts with them (by text, role, label) rather than testing internal implementation details.

```
code
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('increments count on button click', () => {
  render(<Counter />);
  const button = screen.getByText('+1');
  fireEvent.click(button);
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

◆ Common Queries & Matchers

Query	Use
getByText	Find element by visible text
getByRole	Find by ARIA role (button, textbox...)
getByLabelText	Find form input by its label
getByTestId	Find via data-testid attribute
toBeInTheDocument()	Assert element is rendered

Query	Use
toHaveTextContent()	Assert element's text

CH 21

Deployment

◆ **Build for Production**

```
code
npm run build    # creates an optimized /dist (Vite) or /build (CRA) folder
```

◆ **Deploy to Vercel**

```
code
npm install -g vercel
vercel login
vercel    # deploy from project folder
```

- Auto-detects Vite/CRA/Next.js.
- Free custom subdomain + easy custom domain setup.
- Auto-deploys on every git push (GitHub integration).

◆ **Deploy to Netlify**

- Drag-and-drop the /dist folder on app.netlify.com, or
- Connect your GitHub repo: build command 'npm run build', publish directory 'dist'.

◆ **Docker**

```
code
# Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
code
docker build -t my-react-app .
docker run -p 8080:80 my-react-app
```

★ **Remember:** Always set environment-specific API URLs via .env files (e.g. VITE_API_URL) - never hardcode localhost URLs in production builds.

CH 22

React Interview Questions

A curated, high-yield set of React interview questions grouped by difficulty — covering the core concepts most frequently asked in real interviews, with crisp, exam-ready answers.

◆ **Beginner (15 questions)**

Q1. What is React?

A declarative, component-based JavaScript library for building user interfaces, developed by Facebook/Meta.

Q2. What is JSX?

A syntax extension that lets you write HTML-like code inside JavaScript; compiled to `React.createElement()` calls by Babel.

Q3. What is the Virtual DOM?

An in-memory representation of the real DOM used by React to efficiently calculate and apply minimal updates.

Q4. Difference between props and state?

Props are read-only data passed from parent to child; state is data owned and managed within a component that can change.

Q5. What are keys in React lists and why are they needed?

Unique identifiers for list items that help React's reconciliation algorithm track item identity across re-renders.

Q6. What is a component in React?

An independent, reusable piece of UI, written as a function or class, that returns JSX.

Q7. Difference between functional and class components?

Functional components are plain functions using hooks for state/lifecycle; class components extend `React.Component` and use `this.state` and lifecycle methods.

Q8. What is the purpose of the `render()` method?

In a class component, `render()` returns the JSX describing what should appear on screen.

Q9. What is a Fragment and why use it?

A wrapper (`<>...</>`) that groups multiple children without adding an extra DOM node.

Q10. What is the significance of the `key` prop?

It gives React a stable identity for each item so it can efficiently add, remove, or reorder DOM nodes.

Q11. What is `React.StrictMode`?

A development-only tool that highlights potential problems (deprecated APIs, unsafe lifecycles, double-invokes functions to catch side-effect bugs).

Q12. What is the difference between HTML and JSX?

JSX uses camelCase attributes (`className`, `onClick`), can embed JS expressions with `{}`, and must return a single root element.

Q13. What is ReactDOM used for?

It provides DOM-specific methods, e.g. `createRoot().render()`, to mount a React component tree into the actual browser DOM.

Q14. Can you use 'if' statements directly inside JSX?

No - JSX only accepts expressions; use ternary, `&&`, or move if/else logic above the return statement.

Q15. What is the difference between a controlled and uncontrolled component?

Controlled: form value is driven by React state. Uncontrolled: form value lives in the DOM and is read via a ref.

◆ Intermediate (16 questions)

Q1. What are React Hooks and why were they introduced?

Functions that let function components use state and other React features, introduced to avoid the complexity of classes and enable logic reuse via custom hooks.

Q2. Explain the dependency array in `useEffect`.

It controls when the effect re-runs: omitted = every render, `[]` = only on mount, `[dep]` = whenever any listed dependency changes.

Q3. What is prop drilling and how do you avoid it?

Passing props through many intermediate components that don't need them; solved with Context API or a state management library.

Q4. Difference between `useMemo` and `useCallback`?

`useMemo` memoizes a computed VALUE; `useCallback` memoizes a FUNCTION reference. Both take a dependency array.

Q5. What is `React.memo`?

A higher-order component that skips re-rendering a component if its props haven't changed (shallow comparison).

Q6. What is the Context API used for?

Sharing global data (theme, auth, locale) across the component tree without manually passing props at every level.

Q7. What is reconciliation?

React's diffing algorithm that compares the new virtual DOM tree with the previous one to compute the minimal set of real DOM updates.

Q8. What is React Fiber?

The reconciliation engine (since React 16) that enables incremental rendering, prioritization of updates, and interruption/resumption of work.

Q9. Explain the `useReducer` hook.

An alternative to `useState` for complex state logic; takes a reducer function and initial state, returns `[state, dispatch]`, similar to Redux.

Q10. What is a custom hook?

A reusable function starting with 'use' that calls other hooks, used to extract and share stateful logic between components.

Q11. What is the difference between `useEffect` and `useLayoutEffect`?

`useEffect` runs asynchronously after the browser paints; `useLayoutEffect` runs synchronously before the browser paints - use it only when you must measure/mutate the DOM before paint.

Q12. How do you optimize performance in a React app?

React.memo, useMemo/useCallback, code splitting with lazy/Suspense, virtualization for long lists, avoiding unnecessary re-renders, and profiling with React DevTools.

Q13. What is lazy loading in React?

Deferring the loading of a component (via React.lazy + dynamic import) until it's actually needed, reducing the initial bundle size.

Q14. What are Higher-Order Components (HOC)?

A function that takes a component and returns a new, enhanced component - a pattern for reusing component logic (e.g. withAuth(Component)).

Q15. What is the significance of the 'key' warning in console?

It warns you're rendering a list without unique keys, which can cause incorrect UI updates and hurt performance.

Q16. What is the difference between state and ref?

Updating state triggers a re-render; updating a ref (.current) does NOT trigger a re-render and persists across renders.

◆ Advanced (13 questions)

Q1. How does React's diffing algorithm work?

It compares trees level by level ($O(n)$ heuristic instead of $O(n^3)$): different element types produce a full new subtree; same type elements are updated in place; keyed lists are diffed by key identity.

Q2. What is the difference between Concurrent Rendering and Legacy rendering in React 18?

Concurrent rendering lets React prepare multiple UI versions and interrupt/prioritize rendering (e.g. via startTransition) instead of blocking the main thread synchronously.

Q3. What are React Server Components?

Components that render entirely on the server, send no JS to the client, and can directly access backend resources - reduces bundle size (used heavily in Next.js App Router).

Q4. Explain the difference between SSR, SSG, and CSR.

CSR: browser renders everything via JS after download. SSR: server renders HTML per-request. SSG: HTML is pre-built at build time and served statically.

Q5. What is hydration in React?

The process where React attaches event listeners and reconciles server-rendered static HTML with the client-side Virtual DOM, making it interactive.

Q6. How would you prevent unnecessary re-renders in a deeply nested component tree?

React.memo on leaf components, useMemo/useCallback for stable references, splitting context to avoid broad re-renders, and colocating state closer to where it's used.

Q7. Explain error boundaries.

Class components implementing componentDidCatch/getDerivedStateFromError that catch JS errors in their child tree and render a fallback UI instead of crashing the whole app.

Q8. What is the difference between useMemo, memo, and PureComponent?

useMemo memoizes a value inside a component; React.memo memoizes a whole functional component's output based on props; PureComponent is the class-component equivalent, doing a shallow prop/state comparison.

Q9. How does batching work in React 18?

React 18 automatically batches multiple state updates (even inside promises, timeouts, native event handlers) into a single re-render, unlike React 17 which only batched inside React event handlers.

Q10. What is startTransition used for?

Marking a state update as non-urgent so React can interrupt it in favor of more urgent updates (like typing), improving perceived responsiveness.

Q11. What are portals in React?

createPortal lets you render a child into a DOM node outside the parent component's DOM hierarchy - useful for modals, tooltips, and dropdowns.

Q12. Explain the useImperativeHandle hook.

Used with forwardRef to customize the ref instance exposed to parent components, restricting what the parent can call on a child.

Q13. How does React handle synthetic events internally?

React attaches a single event listener at the root and delegates events, wrapping native events in a cross-browser SyntheticEvent object pooled for performance (pooling removed in React 17+).

◆ Scenario Based (6 questions)

Q1. A list re-renders every item when only one changes - how do you fix it?

Check that stable, unique keys are used (not array index if list order changes); wrap list item components in React.memo; ensure the parent isn't passing new inline object/function props each render.

Q2. An input loses focus after every keystroke - why, and how do you fix it?

Usually the input component is being re-created (e.g. defined inside the parent's render, or the key changes) - move the component definition outside, or check the key prop isn't changing.

Q3. useEffect runs twice on mount in development - is this a bug?

No - React 18 StrictMode intentionally double-invokes effects (mount, unmount, mount) in dev to help catch missing cleanup logic; it does not happen in production.

Q4. How would you share state between two sibling components?

Lift the state up to their closest common parent and pass it down via props, or use Context/a global store if many components need it.

Q5. Your app's bundle size is too large - what would you check?

Analyze the bundle (source-map-explorer), apply route-based code splitting with lazy/Suspense, remove unused dependencies, use tree-shakeable imports, and lazy-load heavy libraries.

Q6. A component fetches data using fetch() inside the component body directly (not in useEffect) - what's wrong?

This causes an infinite fetch loop since fetching triggers a state update which re-renders the component and calls fetch again; the fetch must go inside useEffect with a proper dependency array.

CH 23

Coding Practice Questions

Hands-on coding exercises with working solutions — practice building these from scratch before checking the code below.

Q1. Build a Counter with increment, decrement, and reset.

```
code
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <button onClick={() => setCount(c => c - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Q2. Toggle a boolean (show/hide password).

```
code
function PasswordField() {
  const [show, setShow] = useState(false);
  return (
    <div>
      <input type={show ? 'text' : 'password'} />
      <button onClick={() => setShow(s => !s)}>{show ? 'Hide' : 'Show'}</button>
    </div>
  );
}
```

Q3. Debounced search input.

```
code
function SearchBox() {
  const [query, setQuery] = useState('');
  const [debounced, setDebounced] = useState('');

  useEffect(() => {
    const t = setTimeout(() => setDebounced(query), 500);
    return () => clearTimeout(t);
  }, [query]);

  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}
```

Q4. Fetch and display a list with loading/error states.

code

```
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(setUsers)
      .catch(e => setError(e.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Q5. Simple to-do list: add, delete, mark complete.

code

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  function addTodo() {
    if (!text.trim()) return;
    setTodos(prev => [...prev, { id: Date.now(), text, done: false }]);
    setText('');
  }

  function toggle(id) {
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t));
  }

  function remove(id) {
    setTodos(prev => prev.filter(t => t.id !== id));
  }

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      <ul>
        {todos.map(t => (
          <li key={t.id} style={{ textDecoration: t.done ? 'line-through' : 'none' }}>
            <span onClick={() => toggle(t.id)}>{t.text}</span>
            <button onClick={() => remove(t.id)}>x</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Q6. Custom hook: useLocalStorage.

code

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}
```

Q7. Custom hook: useFetch.

code

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let ignore = false;
    setLoading(true);
    fetch(url).then(r => r.json()).then(d => {
      if (!ignore) { setData(d); setLoading(false); }
    });
    return () => { ignore = true; };
  }, [url]);

  return { data, loading };
}
```

Q8. Parent-child communication: child updates parent state.

code

```
function Parent() {
  const [message, setMessage] = useState('');
  return <Child onSend={msg => setMessage(msg)} />;
}

function Child({ onSend }) {
  return <button onClick={() => onSend('Hello from child')}>Send</button>;
}
```

Q9. Implement a simple accordion.

```
code
function Accordion({ items }) {
  const [openIndex, setOpenIndex] = useState(null);
  return (
    <div>
      {items.map((item, i) => (
        <div key={i}>
          <h4 onClick={() => setOpenIndex(openIndex === i ? null : i)}>
            {item.title}
          </h4>
          {openIndex === i && <p>{item.content}</p>}}
        </div>
      ))}
    </div>
  );
}
```

Q10. Pagination component.

```
code
function Pagination({ totalItems, itemsPerPage, onPageChange }) {
  const totalPages = Math.ceil(totalItems / itemsPerPage);
  const [page, setPage] = useState(1);

  function goTo(p) {
    setPage(p);
    onPageChange(p);
  }

  return (
    <div>
      {Array.from({ length: totalPages }, (_, i) => i + 1).map(p => (
        <button key={p} disabled={p === page} onClick={() => goTo(p)}>{p}</button>
      ))}
    </div>
  );
}
```

Q11. Modal component using a portal.

```
code
function Modal({ children, onClose }) {
  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body
  );
}
```

Q12. Form validation without a library.

code

```
function SignupForm() {
  const [email, setEmail] = useState('');
  const [error, setError] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    if (!/^\\S+@\\S+\\.\\S+$/\\.test(email)) {
      setError('Enter a valid email');
      return;
    }
    setError('');
    // submit logic
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <button type="submit">Sign up</button>
    </form>
  );
}
```

CH 24

Mini Projects

Five portfolio-ready projects that combine everything from this notebook. Build them in order — each adds a new layer of complexity.

★ **Todo App**

- Features: add/edit/delete/complete tasks, filter (all/active/completed), persist to localStorage.
- Key concepts: useState, useEffect, custom useLocalStorage hook, controlled inputs, list rendering with keys.
- Stretch goals: drag-and-drop reordering, due dates, categories/tags.

★ **Weather App**

- Features: search a city, show current temperature/conditions, 5-day forecast.
- Key concepts: fetch/axios to a weather API (e.g. OpenWeatherMap), loading/error states, debounced search, env variables for the API key.
- Stretch goals: geolocation-based auto-detect, unit toggle (C/F), favorites list.

★ **Employee Management System**

- Features: list, add, edit, delete employees; connects to a Spring Boot REST API for CRUD.
- Key concepts: React Router for /employees, /employees/new, /employees/:id/edit; controlled forms; axios service layer; loading/error UI.
- Stretch goals: search/filter, pagination, JWT-protected admin routes.

★ **E-Commerce Product UI**

- Features: product grid, product detail page, cart with quantity update, checkout summary.
- Key concepts: Context API or Redux Toolkit for cart state, React Router dynamic routes for product IDs, useMemo for computed totals.
- Stretch goals: filters/sorting, wishlist, persisted cart in localStorage.

★ **Real-time Chat Application**

- Features: join a room, send/receive messages instantly, online user list.
- Key concepts: WebSockets or Socket.IO client, useEffect for subscribing/cleaning up socket listeners, useRef to auto-scroll to the latest message.
- Stretch goals: typing indicators, message timestamps, private messaging.

CH 25

Cheat Sheet

◆ **Hooks Summary**

Item	Meaning / Use
useState(initial)	Local state -> [value, setValue]
useEffect(fn, deps)	Side effects / lifecycle
useContext(Ctx)	Read a Context value
useReducer(reducer, init)	Complex state -> [state, dispatch]
useMemo(fn, deps)	Memoize a computed value
useCallback(fn, deps)	Memoize a function reference
useRef(initial)	Mutable ref / DOM access, no re-render

◆ **Lifecycle (Class <-> Hook equivalent)**

Item	Meaning / Use
componentDidMount	useEffect(fn, [])
componentDidUpdate	useEffect(fn, [deps])
componentWillUnmount	useEffect(() => { return cleanupFn }, [])
shouldComponentUpdate	React.memo(Component)

◆ **Routing Summary**

Item	Meaning / Use
<Route path="/x" element={}<X/>}/>	Define a route
<Link to="/x">	Navigate without reload
useNavigate()	Programmatic navigation
useParams()	Read dynamic URL segments
useSearchParams()	Read/write query string
<Outlet/>	Render matched nested route

◆ **Redux Summary**

Item	Meaning / Use
configureStore({reducer})	Create the store
createSlice({name, initialState, reducers})	Define state + actions + reducer
useSelector(fn)	Read state from the store
useDispatch()	Get dispatch function
createAsyncThunk	Handle async logic (API calls)

◆ **Common Interview Tricks**

Item	Meaning / Use
State updates are asynchronous	Don't rely on state immediately after calling setState
Batching	Multiple setState calls in one handler = one re-render (React 18: everywhere)
Keys must be stable & unique	Avoid array index as key for dynamic lists
useEffect cleanup	Return a function to prevent memory leaks (listeners, timers, subscriptions)
Props are read-only	Never mutate props directly in a child

■ **End of Notes — Happy coding & good luck with your interviews!**